Informe de diseño y sincronización

Torneo de Komikan Concurrente • Gabriela Esparza y Jose Larrondo

1. Diseño del esquema de sincronización

1.1. Organización de las partidas. El programa utiliza hilos POSIX dentro de un mismo proceso. El servidor recibe a los jugadores, forma parejas según el orden en que llegan y organiza partidas independientes, sin eliminatorias. Se exige que la cantidad de jugadores, N, sea par y esté entre 2 y 128; el máximo de 128 es una decisión de implementación para limitar el uso de recursos. Pueden jugar hasta M mesas a la vez, aunque se hayan solicitado más mesas que partidas. **Cada partida activa utiliza 14 hilos**: uno para el Puma, doce para los Perros y uno para el Árbitro, además de los hilos del servidor. Cada partida tiene su propio tablero y sus recursos de sincronización. Antes de comenzar, los 14 hilos esperan una señal de inicio común para evitar que una ficha se mueva mientras la partida todavía se prepara.

1.2. Interpretación del tablero. Se distinguieron las **posiciones** de las **conexiones**: los círculos del dibujo son los lugares donde pueden estar las fichas y las líneas indican por dónde se permite moverlas. El tablero se guarda en una matriz de 7×5, aunque solo **28 posiciones son válidas**: 25 de la cuadrícula y tres del trono. En la cuadrícula solo hay movimientos horizontales y verticales entre círculos vecinos, sin diagonales. Las conexiones del trono se definieron por separado, porque no pueden determinarse solo comparando filas y columnas. El archivo de configuración permite cambiar la posición inicial de las 13 fichas, pero cada una debe ocupar un círculo válido y distinto.

En el trono, la línea horizontal que une los dos círculos superiores cruza una línea vertical. **Ese cruce no es una posición**, porque no tiene círculo. Por lo tanto, ambos círculos laterales están conectados directamente, y el vértice superior del trono, identificado como T, también se conecta directamente con el círculo central de la primera fila de la cuadrícula. La distancia entre los círculos en el dibujo no determina la cantidad de movimientos.

1.3. Movimientos y capturas. Para realizar un movimiento normal se comprueba que ambas posiciones estén conectadas y que el destino esté libre. Los Perros solo pueden realizar este tipo de movimiento. El Puma también puede capturar: salta sobre un Perro y llega a una posición vacía, siguiendo **dos conexiones seguidas en línea recta**. Por eso se definieron las rutas de captura por separado de las conexiones de movimiento. Por ejemplo, se permite saltar desde T, sobre el círculo superior izquierdo del trono, hasta la esquina superior izquierda de la cuadrícula; en cambio, no se permite cambiar de dirección después de pasar sobre el Perro. También se permite una captura por la línea vertical que baja desde T al centro de la cuadrícula. Los cruces sin círculo no sirven como posiciones intermedias. **Las capturas son opcionales y se realizan de una en una**: si el Puma puede volver a capturar, debe enviar otra solicitud. El Puma gana al capturar seis Perros; los Perros ganan cuando el Puma ya no tiene movimientos ni capturas legales.

1.4. Decisión y arbitraje. Las fichas se mueven sin turnos. Cada hilo espera un tiempo aleatorio dentro del intervalo configurado (50–300 ms por defecto), consulta una copia del estado actual del tablero, obtiene sus movimientos legales y **elige uno al azar**. El Puma considera tanto movimientos normales como capturas, sin dar prioridad obligatoria a estas últimas. Cada ficha utiliza su propio generador `mt19937`, para no compartir un generador aleatorio entre hilos. La semilla puede configurarse, aunque no fija el orden en que el sistema operativo ejecuta las fichas. Cuando una ficha no tiene movimientos disponibles, espera a que cambie el tablero en lugar de revisarlo continuamente. Los retardos usan `pthread_cond_timedwait`, de modo que pueden interrumpirse al finalizar la partida.

La ficha agrega su solicitud a una **cola FIFO** protegida por mutex y espera la respuesta en su propio buzón. Como cada ficha envía una sola solicitud a la vez, puede haber como máximo trece pendientes. Si no hay solicitudes, el Árbitro **queda bloqueado con** `pthread_cond_wait`; cuando llega una, recibe una señal y despierta. Entonces revisa otra vez que el movimiento sea legal, porque el tablero pudo cambiar mientras la solicitud esperaba. Después responde si fue aceptado o rechazado, y despierta a la ficha para que continúe si la partida sigue activa. También se notifica cuando una ficha es capturada o la partida termina. La cola respeta el orden en que llegan las solicitudes, no el momento en que cada ficha decidió moverse.

1.5. Actualización del tablero y finalización. Solo el Árbitro modifica el tablero. Para cada solicitud, mantiene protegido el estado mientras valida el movimiento, cambia las posiciones, retira al Perro capturado si corresponde y comprueba la victoria. Así, si dos fichas quieren ocupar el mismo lugar, la segunda solicitud se rechaza cuando el destino ya está ocupado. La victoria también se revisa al comenzar, por si una configuración inicial deja al Puma sin movimientos. Si un Perro es capturado mientras duerme o espera respuesta, se le notifica para que su hilo termine de forma ordenada. Las parejas que no tienen mesa esperan sin consumir CPU revisando continuamente, y se atienden cuando se libera una. La consola y los registros se sincronizan por separado para no escribir en disco mientras se cambia el tablero. Si un jugador abandona o falla, gana su oponente. Ante `SIGINT`, recibido mediante `sigwait`, se dejan de iniciar partidas, se despiertan los hilos en espera y se liberan los recursos cuando terminan. Si falla la creación de algún hilo o recurso POSIX, se limpian los elementos que alcanzaron a inicializarse.

2. Análisis de riesgos de concurrencia

2.1. Condiciones de carrera. Dos fichas pueden elegir el mismo destino al consultar el tablero casi al mismo tiempo. Si ambas pudieran modificarlo directamente, podrían ocupar la misma posición o dejar una captura incompleta. Para evitarlo, **solo el Árbitro valida y aplica los movimientos**, con el tablero protegido por un mutex. También comprueba la victoria antes de aceptar nuevos movimientos. Como cada mesa tiene su propio tablero, los movimientos de una partida no afectan a las demás.

2.2. Deadlock. Podría ocurrir un bloqueo mutuo si una ficha esperara la respuesta del Árbitro sin liberar el mutex que este necesita para contestar. Se utiliza `pthread_cond_wait`, que libera el mutex mientras el hilo espera y vuelve a tomarlo al despertar. Así, el hilo no conserva ese bloqueo mientras espera una respuesta, lo que evita una posible espera circular entre la ficha y el Árbitro. Además, no se usan mutexes separados para cada casilla ni se espera el término de otros hilos con `pthread_join` mientras se mantiene bloqueado el tablero. Cuando finaliza la partida, se notifica a las fichas y al Árbitro para evitar que queden esperando respuestas que ya no llegarán.

2.3. Livelock. Podría haber actividad sin avances si varias fichas intentaran una y otra vez movimientos que dejan de ser válidos por los cambios del tablero. En este diseño, **cada solicitud recibe una respuesta**. Si se rechaza, la ficha espera su siguiente ciclo y vuelve a consultar los movimientos legales. Si no tiene opciones, queda bloqueada hasta que cambie el tablero. Esto reduce los intentos inútiles, pero no garantiza que una partida termine. Las fichas podrían seguir realizando movimientos legales sin que el Puma capture seis Perros ni quede atrapado. Como no existe un límite de tiempo o movimientos, la partida podría continuar hasta que se interrumpa el torneo.

2.4. Starvation. Una ficha podría tener menos oportunidades de moverse si sus retardos son mayores o el sistema operativo ejecuta más seguido a otros hilos. La **cola FIFO mantiene el orden de las solicitudes que ya ingresaron**, pero no puede asegurar que todas las fichas lleguen a la cola con la misma frecuencia ni que sus movimientos sean aceptados. Del mismo modo, las parejas pendientes reciben una mesa cuando se libera una, pero el programa no controla por completo el orden en que el sistema operativo ejecuta los hilos.

3. Evaluación y justificación del diseño

3.1. Complejidad. Se decidió que **solo el Árbitro actualice el tablero**, en lugar de permitir que las trece fichas lo cambien directamente. Esto requiere una cola, buzones y señales de sincronización, pero facilita comprobar que cada movimiento y captura se realice correctamente. También se prefirió una matriz y tablas de conexiones antes que un *bitboard*, porque permiten representar con claridad las posiciones del trono y las rutas de captura.

3.2. Rendimiento y latencia. Como el Árbitro procesa una solicitud a la vez, cada movimiento debe esperar su turno y su respuesta. Esto agrega **latencia** y limita la cantidad de movimientos que pueden completarse por unidad de tiempo (**throughput**) dentro de una partida. A cambio, elegir entre movimientos legales evita enviar propuestas claramente inválidas; las esperas bloqueantes evitan revisar continuamente las colas y los registros se escriben fuera de la actualización del tablero. Estos efectos se explican por el diseño y no corresponden a mediciones de rendimiento.

3.3. Concurrencia permitida. Aunque el Árbitro aplica los movimientos uno por uno dentro de cada partida, **las trece fichas toman decisiones y esperan de forma independiente**. Además, las mesas pueden funcionar al mismo tiempo, hasta el límite M, porque cada una tiene su propio tablero y Árbitro. Se prefirió mantener un estado consistente y un cierre ordenado antes que permitir varios cambios simultáneos sobre el mismo tablero. Las decisiones sobre cruces sin círculo, capturas rectas, movimientos aleatorios y capturas individuales se tomaron para resolver aspectos que el enunciado no detalla completamente, sin incorporar reglas de otras variantes del juego.